

Guía-Taller: Persistencia de Archivos con Python

Ing. Jonathan Torres, PhD.
Docente - Programación Avanzada
Proyecto Curricular de Ingeniería en Sistemas
Universidad Distrital Francisco José de Caldas
jrtoresc@udistrital.edu.co

30 de abril del 2025

Introducción

Una de las características más útiles de los programas informáticos es su capacidad para almacenar información de manera persistente, es decir, que los datos se mantengan disponibles incluso después de cerrar el programa. En esta guía-taller abordaremos el tema de **persistencia de archivos** utilizando el lenguaje de programación Python.

La persistencia se logra mediante el uso de archivos que permiten guardar y recuperar información. Esto es fundamental en aplicaciones como agendas, sistemas de gestión de usuarios, bases de datos simples, entre otros.

A través de esta guía construirás paso a paso un programa en consola que permite crear, editar, listar y eliminar notas personales. Cada paso está acompañado de explicaciones conceptuales y fragmentos de código comentados para que puedas comprender tanto el *qué* como el *por qué* de cada línea.

1. Objetivos de Aprendizaje

- Comprender el concepto de persistencia de datos y su implementación mediante archivos planos.
- Manejar operaciones básicas de archivos en Python: lectura, escritura, edición y eliminación.
- Aplicar buenas prácticas de programación, como modularización del código y validación de entradas.
- Utilizar módulos estándar como `os`, `datetime` y `shutil`.

2. ¿Qué es la Persistencia de Archivos?

La persistencia en programación se refiere a la capacidad de un programa para mantener sus datos más allá de la ejecución actual. En este taller usaremos archivos de texto para lograrlo, escribiendo datos con la función `open()` de Python y manipulando rutas y carpetas con el módulo `os`.

3. Preparación del Proyecto

Vamos a crear la estructura básica del proyecto. Abre VS Code y ejecuta los siguientes comandos en la terminal, según el sistema operativo que estés utilizando:

Para Linux o macOS

```
mkdir gestor_notas
cd gestor_notas
mkdir notas
touch main.py utils.py
```

Para Windows (PowerShell)

```
mkdir gestor_notas
cd gestor_notas
mkdir notas
New-Item main.py -ItemType File
New-Item utils.py -ItemType File
```

- `notas/`: Contendrá las notas de texto creadas.
- `main.py`: Archivo principal del programa con el menú.
- `utils.py`: Módulo con funciones auxiliares.

4. Creación de una Nota

Comenzamos con la función que permite crear una nota y guardarla como archivo `.txt`:

```
# utils.py
import os
from datetime import datetime

def crear_nota(nombre, contenido):
    fecha = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    ruta = os.path.join("notas", f"{nombre}.txt")
    with open(ruta, "w", encoding="utf-8") as archivo:
        archivo.write(f"Fecha: {fecha}\n\n")
        archivo.write(contenido)
```

¿Qué hace este código?

- Usa `datetime.now()` para registrar la fecha y hora de creación.
- Construye la ruta del archivo combinando la carpeta y el nombre.
- Abre el archivo en modo escritura (`"w"`). Si el archivo ya existe, se sobrescribirá.
- Escribe la fecha y el contenido en el archivo.

5. Lectura del contenido de una nota

Ahora vamos a leer el contenido de una nota existente.

```
def leer_nota(nombre):
    ruta = os.path.join("notas", f"{nombre}.txt")
    if os.path.exists(ruta):
        with open(ruta, "r", encoding="utf-8") as archivo:
            print(archivo.read())
    else:
        print("Nota no encontrada.")
```

Puntos clave:

- Verificamos que el archivo exista con `os.path.exists()`.
- Abrimos el archivo en modo lectura (`"r"`).
- Mostramos su contenido por consola.

6. Listar todas las notas

```
def listar_notas():
    for archivo in os.listdir("notas"):
        if archivo.endswith(".txt"):
            print(f"- {archivo[:-4]}")
```

¿Qué hace esto?

- Lista todos los archivos en la carpeta `notas`.
- Filtra aquellos con extensión `.txt`.
- Muestra los nombres sin la extensión.

7. Buscar una palabra dentro de las notas

```
def buscar_palabra(clave):
    encontrados = []
    for archivo in os.listdir("notas"):
        ruta = os.path.join("notas", archivo)
        with open(ruta, "r", encoding="utf-8") as f:
            if clave.lower() in f.read().lower():
                encontrados.append(archivo)
    print("Coincidencias encontradas:")
    for n in encontrados:
        print(f" - {n[:-4]}")
```

Explicación:

- Lee cada nota y busca si la palabra clave aparece.
- No distingue mayúsculas o minúsculas.
- Imprime la lista de coincidencias encontradas.

8. Editar una nota (creando respaldo automático)

```
import shutil

def editar_nota(nombre, nuevo_contenido):
    ruta = os.path.join("notas", f"{nombre}.txt")
    if os.path.exists(ruta):
        respaldo = os.path.join("notas", f"{nombre}_bak.txt")
        shutil.copy(ruta, respaldo)
        with open(ruta, "w", encoding="utf-8") as archivo:
            archivo.write(nuevo_contenido)
        print("Nota actualizada y respaldo creado.")
    else:
        print("Nota no encontrada.")
```

¿Qué estamos haciendo aquí?

- Creamos una copia de respaldo antes de sobrescribir.
- Luego reescribimos el archivo con el nuevo contenido.

9. Eliminar una nota

```
def eliminar_nota(nombre):
    ruta = os.path.join("notas", f"{nombre}.txt")
    if os.path.exists(ruta):
        os.remove(ruta)
        print("Nota eliminada.")
    else:
        print("Nota no encontrada.")
```

10. Menú Principal del Programa

```
# main.py
from utils import *

while True:
    print("\n--- Gestor de Notas ---")
    print("1. Crear nota")
    print("2. Leer nota")
    print("3. Listar notas")
    print("4. Buscar palabra")
    print("5. Editar nota")
    print("6. Eliminar nota")
    print("7. Salir")

    opcion = input("Seleccione una opción: ")

    if opcion == "1":
        nombre = input("Nombre de la nota: ")
        contenido = input("Contenido: ")
        crear_nota(nombre, contenido)
    elif opcion == "2":
        nombre = input("Nombre de la nota: ")
        leer_nota(nombre)
    elif opcion == "3":
        listar_notas()
    elif opcion == "4":
        clave = input("Palabra clave: ")
        buscar_palabra(clave)
    elif opcion == "5":
        nombre = input("Nombre de la nota: ")
        nuevo = input("Nuevo contenido: ")
        editar_nota(nombre, nuevo)
    elif opcion == "6":
        nombre = input("Nombre de la nota: ")
        eliminar_nota(nombre)
    elif opcion == "7":
        break
    else:
        print("Opción no válida.")
```

Reflexión Final

Este taller te ha guiado por los conceptos esenciales del manejo de archivos en Python y su aplicación práctica. A través de un proyecto realista, aprendiste a usar funciones clave para leer, escribir, editar y eliminar archivos. Este conocimiento es esencial para aplicaciones que requieren persistencia local de datos sin el uso de bases de datos.

Actividades para la clase

Actividad 1: Comentar el Código

Objetivo: Mejorar la comprensión del código mediante comentarios explicativos.

Indicaciones:

- Lee todo el código del proyecto.
- En cada función y bloque de código, agrega comentarios que expliquen qué hace el código. Por ejemplo, explica el propósito de cada función (`crear_nota`, `leer_nota`, etc.) y cómo funcionan los módulos que se importan, como `os`, `shutil`, `datetime`.
- Los comentarios deben ser claros y concisos. Describe lo que hace cada línea o conjunto de líneas en términos sencillos.
- Añade comentarios sobre las decisiones de diseño: ¿por qué se usa `with open()` en lugar de simplemente `open()`? ¿Por qué se usa `os.path.exists()` antes de leer o escribir un archivo?
- Asegúrate de que todos los métodos, variables y bloques de código tengan al menos un comentario explicativo.

Actividad 2: Validación de Entrada de Datos

Objetivo: Implementar validaciones de entrada para mejorar la robustez del programa.

Indicaciones:

- Localiza las funciones de creación y edición de notas: `crear_nota()` y `editar_nota()`.
- Antes de guardar el archivo, agrega un bloque de código que verifique si el nombre de la nota está vacío. Usa `if not nombre:` para comprobarlo y, si es vacío, muestra un mensaje pidiendo al usuario que ingrese un nombre válido.
- Para la validación del contenido de la nota, usa una comprobación similar para asegurarte de que el contenido tenga más de 10 caracteres. Si no es así, muestra un mensaje indicando que el contenido debe ser más largo.
- Usa un bucle `while` para seguir pidiendo la entrada al usuario hasta que la validación sea exitosa.
- Ejemplo de validación de nombre de la nota:

```
while not nombre:
    nombre = input("El nombre de la nota no puede estar vacío. Ingresa un nombre válido: ")
```

Actividad 3: Crear Función de Contador de Notas

Objetivo: Contar cuántas notas existen en la carpeta y mostrarlas por consola.

Indicaciones:

- Crea una nueva función llamada `contar_notas()`.
- Usa `os.listdir("notas")` para obtener la lista de todos los archivos en la carpeta `notas/`.
- Filtra los archivos para que solo cuente aquellos con extensión `.txt`.
- Utiliza `len()` para contar el número de archivos y muestra el resultado con un mensaje como "Número de notas guardadas: X".
- Ejemplo:

```
def contar_notas():
    archivos = [archivo for archivo in os.listdir("notas") if archivo.
endswith(".txt")]
    print(f"Número de notas guardadas: {len(archivos)}")
```

Actividades adicionales sugeridas

- Añadir función de exportación de notas a PDF.
- Guardar cada nota con una estructura de carpeta **año/mes**.
- Incorporar manejo de múltiples usuarios.
- Implementar autenticación con contraseña para cada usuario.

Evaluación

- Implementación completa del sistema: 40 %
- Modularización y uso de funciones: 20 %
- Validación de entradas y manejo de errores: 20 %
- Creatividad y funcionalidades adicionales: 20 %